Faster Vehicle Routing: Solving CVRPTW with OCI AI Accelerator Pack

By Patrick Leung, Dennis Kennetz, and Ainura Djumagulov

The best logistics teams are effectively invisible. Success is measured by the problems customers never see: the right van arriving at the right doorstep within the promised 30-minute slot without wasted miles. At enterprise scale, that "invisible" optimisation shows up in route productivity, labour hours, On Time in Full (OTIF), and carbon footprint.

In this post, we tackle a common real world routing challenge, the Capacitated Vehicle Routing Problem with Time Windows (CVRPTW), using the OCI AI Accelerator Pack on Oracle Cloud Infrastructure (OCI). We start with a practical business scenario, then validate results against academic benchmarks and published best known solutions.

Business scenario: "Eden Logistics" in London

Eden Logistics is a fictional delivery operator and plans daily routes across London.

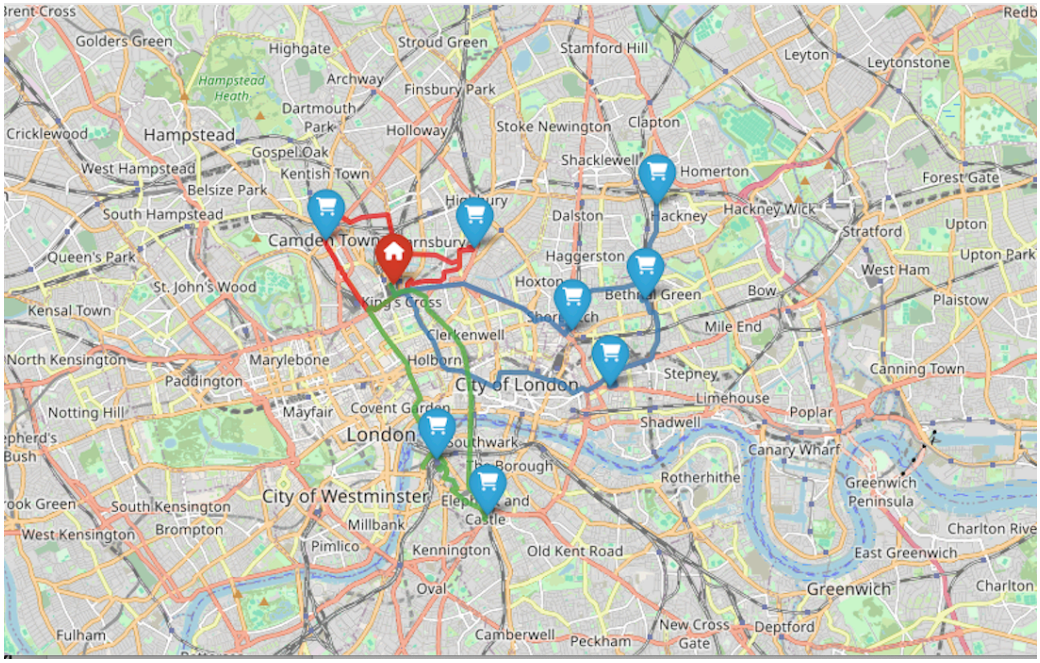

Figure 1: Eden Logistics delivery route

Eden Logistics must build routes for a fleet of vehicles that:

1. Start and end at a depot
2. Serve every customer exactly once
3. Respect vehicle capacity constraints such as parcel count, weight, or volume
4. Respect time windows

a. Minimise total travel cost, such as distance or travel time

This is operationally important because time windows are a direct proxy for customer experience and Service Level Agreement (SLA). As customer count and constraints increase, the number of feasible combinations grows extremely quickly, so high-quality heuristics and acceleration become essential. Another common scenario is routing from drivers' homes instead of a depot. This scenario is also supported, but we focus on a depot in the remainder of this post.

OCI AI Accelerator Pack

This solution deploys a reference architecture into the customer's OCI tenancy to support secure, scalable application delivery. It includes:

Networking: A Virtual Cloud Network (VCN) with configurable connectivity options, including public, restricted, or private addressing, to align with enterprise security and compliance requirements.

Container platform: Oracle Kubernetes Engine (OKE) to run and manage the service components with a resilient, production ready architecture.

Deployment and operations: An AI Blueprint to standardise application deployment and support operational visibility through monitoring.

GPU accelerated optimisation: GPUs for the cuOpt VRP server, orchestrated by OKE. Example sizing options include:

- PoC: VM.GPU.A10.2
- Small: BM.GPU4.8
- Medium: BM.GPU.A100.v2.8

This stack is designed to support a clear path from proof of concept to production by using repeatable deployment patterns and scalable infrastructure on OCI.

Step-by-step guide

If you are interested in the technical details, work through the Jupyter notebook hosted on GitHub. It walks through each step and includes options to test different scenarios or use your own data.

At a high level, the workflow is:

1. Deploy NVIDIA cuOpt using the OCI AI Accelerator Pack

2. Prepare a cost matrix that represents travel cost between every pair of locations
3. Define fleet data, including vehicle start and end locations and capacities
4. Define task data, including customer locations, demands, time windows, and service times
5. Submit the problem to cuOpt through a REST API and retrieve the resulting routes

Benchmark: Gehring and Homberger, 200 customers

To ground the discussion, we use two Gehring and Homberger VRPTW instances with 200 customers each. We use the common benchmark objective: first minimising the number of vehicles reduces fixed cost (drivers, shifts), then minimising total travel time reduces variable cost, empty miles, and emissions.

The first benchmark is C1_2_1, where C denotes clustered customer locations. Customers are grouped into distinct geographic clusters rather than being spread uniformly across the service area. Figure 2 plots the dataset in X and Y coordinates and highlights the time window constraints associated with each delivery, meaning the interval within which service must begin. This illustrates both the spatial clustering and the scheduling constraints that make CVRPTW challenging.

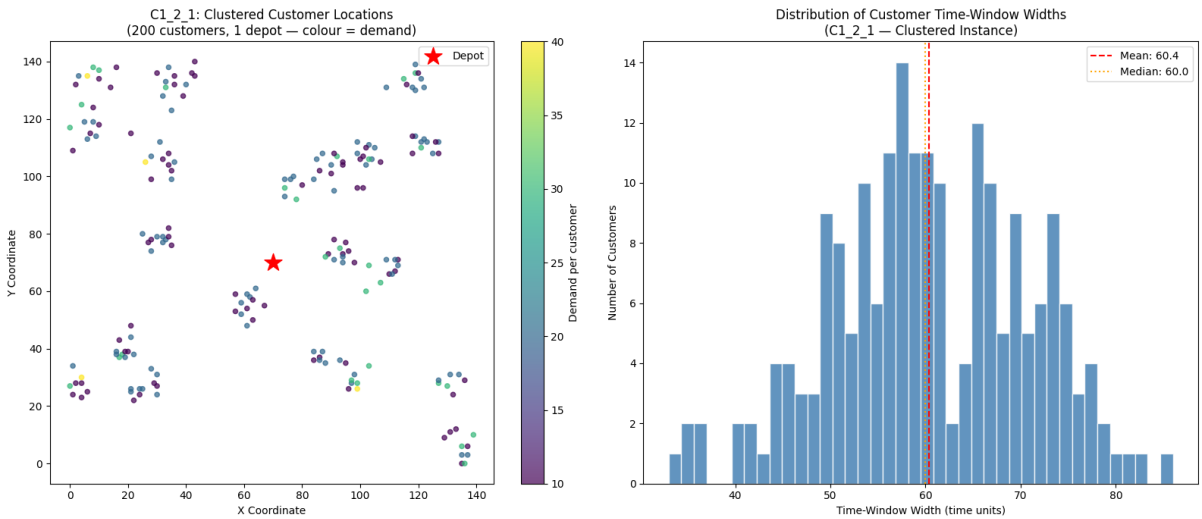

Figure 2: (a) C1_2_1 clustered customer locations (b) Distribution of customer time windows

The second benchmark is R1_2_1, where R denotes random customer locations. Customers are spread more uniformly and unpredictably across the service area rather than forming tight geographic clusters. R1_2_1 is harder because random spatial distribution removes the natural cluster structure that makes good routes

more obvious, increasing inter-stop travel and making tight time windows more difficult to satisfy without additional detours or waiting.

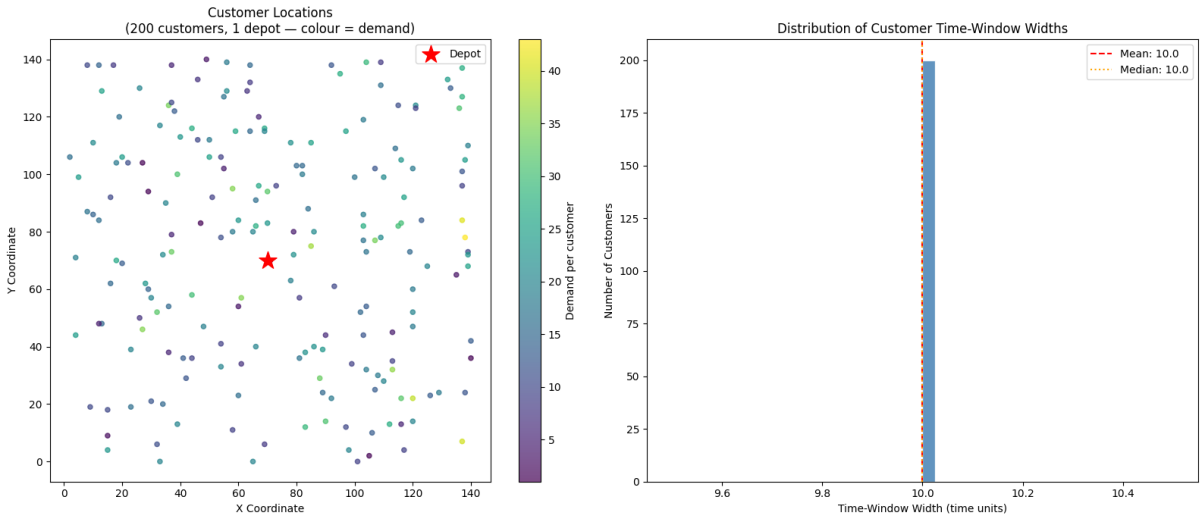

Figure 3: (a) R1_2_1 Random customer locations (b) Distribution of customer time windows

Both benchmark instances have published best-known solutions, including the minimum fleet size and the corresponding best-known total cost. This makes it straightforward to quantify cuOpt's solution quality by measuring the vehicle gap and cost gap against these reference values.

The cost values in these benchmarks are derived from Euclidean distances based on the provided coordinates, rather than real-world road network travel times. For more accurate routing, you can replace the Euclidean cost matrix with travel times or distances derived from a road network model.

Benchmark Results

Part 1: C1_2_1 clustered benchmark

Below are the outcomes from two solver time budgets: 5 seconds and 10 seconds. Table 1 summarises results for the C1_2_1 clustered benchmark.

Instance	Distribution	Time limit	Vehicles used	Best-known vehicles	Cost	Best-known cost	Cost gap
C1_2_1	Clustered	5	20	20	2704.57	2704.57	0.00%
C1_2_1	Clustered	10	20	20	2704.57	2704.57	0.00%

Table 1: C1_2_1 benchmark results

The exact set of routes produced with the 10-second time budget is shown in Figure 4.

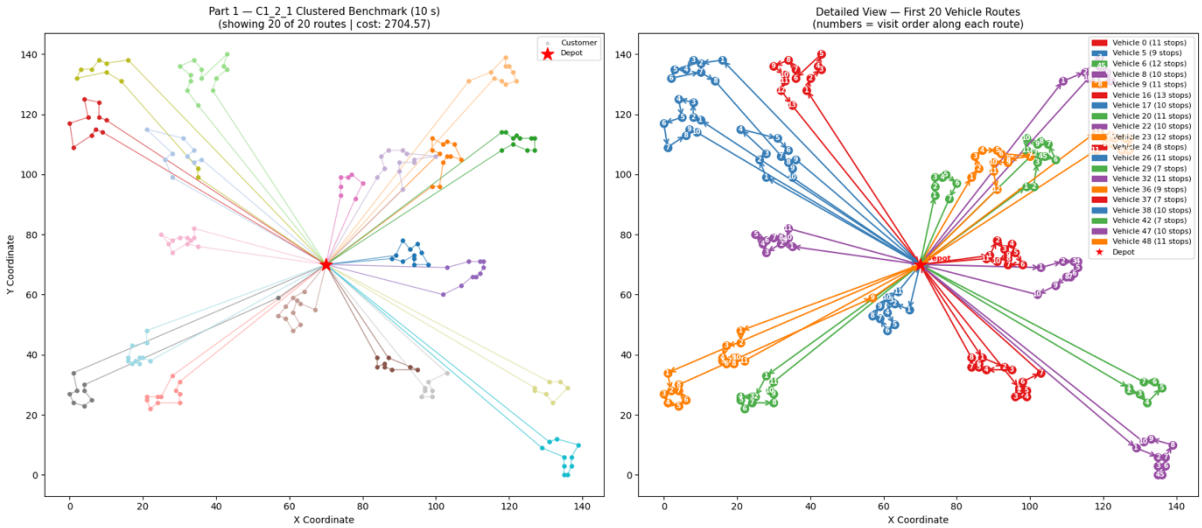

Figure 4. (a) C1_2_1 clustered benchmark (10s) (b) Detailed view of the first 20 vehicle routes

With a 5 second time limit, we found a plan that matches the best-known solution exactly. With a 10 second time limit, there is no further improvement in cost, indicating that we have already reached the most optimal solution in the 5 second time budget.

Best known solutions can typically take hours or even days to compute, and they represent the strongest results reported by academic researchers across a range of algorithms and significant computational resources. Matching the published best-known result within a 5-second budget demonstrates that GPU-accelerated heuristics can reach benchmark-leading quality under operational runtimes.

Part 2: R1_2_1 Random benchmark

Now we turn to the R1_2_1 random benchmark, which is more challenging due to the lack of natural geographic clusters. Below are the outcomes from two solver time budgets: 5 seconds and 10 seconds. Table 2 summarises cuOpt's results on the R1_2_1 random benchmark.

Instance	Distribution	Time limit	Vehicles used	Best-known vehicles	Cost	Best-known cost	Cost gap
R1_2_1	Random	5	20	20	4879.69	4784.11	2.00%
R1_2_1	Random	10	20	20	4824.37	4784.11	0.84%

Table 2: R1_2_1 benchmark results

The exact set of routes produced with the 10-second time budget is shown in Figure 5.

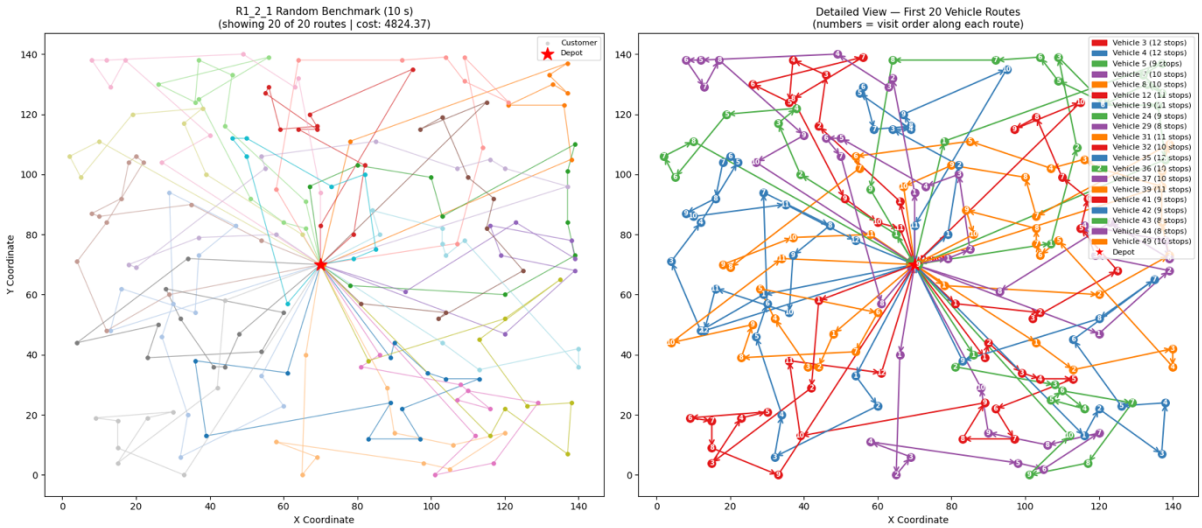

Figure 5 (a) R1_2_1 random benchmark (10 seconds) (b) Detailed view of the first 20 vehicle routes

Note that this dataset is more interesting from an academic perspective, as customers are likely to be clustered in residential areas rather than randomly scattered over the city.

This run reaches an operationally high-quality answer in just 5 seconds by matching the best-known fleet size of 20 vehicles. This is often the primary requirement because fleet size drives the largest fixed overhead (drivers, shifts, and dispatch capacity). Extending the time limit to 10 seconds improves route efficiency (cost drops from 4,879.69 to 4,824.37, narrowing the gap to the best-known cost from 2.00% to 0.84%), but the incremental savings typically have less business impact than getting the vehicle count right.

In a real-world business setting, chasing a mathematically proven optimum often leads to diminishing returns. While academic benchmarks focus on finding the single most efficient solution regardless of computation time, in practice it is usually more realistic to establish a baseline based on existing business processes and then measure improvements in fleet size, SLA adherence, and travel cost under tight time limits.

That said, you have full control over this trade-off by adjusting the solve time to balance speed and incremental optimisation. For example, in a dynamic routing scenario with real-time traffic updates, you may prioritise faster runtimes to quickly adapt routes, even if it means accepting a slightly higher cost. Conversely, for static planning where routes are set well in advance, you might allow for longer runtimes to achieve the lowest possible cost. The flexibility of the solution allows you to adjust this trade-off based on your operational needs.

Conclusion

In this post, we showed how to solve the Capacitated Vehicle Routing Problem with Time Windows (CVRPTW) using NVIDIA cuOpt deployed with the OCI AI Accelerator Pack. We started from a practical enterprise routing scenario and then validated solution quality on two Gehring and Homberger benchmark instances, C1_2_1 (clustered) and R1_2_1 (random).

Across both benchmarks, cuOpt delivered near optimal results in seconds under tight solver time budgets. On the clustered C1_2_1 instance, it matched the published best-known solution within 5 seconds, reaching the best-known fleet size and cost. On the more challenging random R1_2_1 instance, it matched the best-known fleet size within 5 seconds and reduced the cost gap further with a 10 second solve, narrowing the gap to under 1 percent. These results demonstrate that high quality routing plans can be generated quickly enough to support operational decision making, not just offline optimisation.

The accompanying Jupyter notebook provides a repeatable way to reproduce the results, experiment with solver time limits, and extend the analysis with additional benchmarks or your own datasets. The OCI AI Accelerator Pack provides a production-oriented foundation for running GPU accelerated optimisation on Oracle Cloud Infrastructure, enabling teams to move from proof of concept to deployment with consistent architecture patterns. Whether you are optimising last mile delivery routes, field service schedules, or other logistics workflows, OCI offers powerful tools that can take Kaizen to the next level. Happy optimising!